

Análise do Impacto de um Pré-Processamento Simétrico $O(n)$ na Redução de Inversões e Eficiência de Algoritmos de Ordenação Adaptativos e Não Adaptativos

Samuel da Penha Nascimento ¹, Francisco das Chagas Rocha ¹

¹Coordenação do Curso de Sistemas de Computação, Universidade Estadual do Piauí
Campus Prof. Alexandre Alves de Oliveira
Parnaíba, Piauí - Brasil

samuelnascimento@aluno.uespi.br, rocha@phb.uespi.br

Abstract. *In resource-constrained systems, quadratic sorting algorithms remain essential due to low memory footprint, yet high inversion counts severely degrade their physical hardware performance. This paper investigates if a low-cost pre-processing technique can reduce inversions such that $C_{pre} + C_{sort} < C_{original}$. We analyze a linear-time $O(n)$ method based on symmetric swaps and adjacent corrections. Five algorithms were benchmarked across six vector topologies ($n \leq 20,000$). The technique reduced inversions by up to 100% at ≈ 1.2 ns/element, achieving speedups up to 99.9% in adaptive algorithms while remaining ineffective for quasi-linear methods, outlining its practical viability limits.*

Resumo. *Em sistemas de recursos restritos, algoritmos quadráticos são vitais pelo baixo consumo de memória, mas a alta quantidade de inversões degrada seu desempenho no hardware. Este trabalho investiga se um pré-processamento de baixo custo reduz inversões viabilizando $C_{pre} + C_{sort} < C_{original}$. Avalia-se um método linear $O(n)$ de trocas simétricas e correções adjacentes. Foram comparados cinco algoritmos em seis topologias de vetores ($n \leq 20.000$). O método reduziu inversões em até 100% a $\approx 1,2$ ns/elemento, gerando ganhos de até 99,9% em algoritmos adaptativos e impacto marginal nos quasilineares, delimitando sua viabilidade prática.*

1. Introdução

A ordenação de dados é uma operação fundamental na Ciência da Computação, sendo essencial em sistemas de busca, bancos de dados e aplicações em tempo real [Knuth 1973, Sedgewick and Wayne 2011]. Embora métodos com complexidade assintótica $O(n \log n)$, como MergeSort e QuickSort, sejam preferíveis para o tratamento de grandes volumes de dados, algoritmos de natureza quadrática $O(n^2)$, como InsertionSort, SelectionSort e BubbleSort, permanecem relevantes em cenários específicos [Cormen et al. 2022]. Essa permanência está relacionada à simplicidade estrutural, ao baixo consumo de memória por operarem *in-place* e à eficiência prática em conjuntos de dados pequenos ou parcialmente ordenados [Cormen et al. 2022, Ziviani 2021].

A eficiência prática de um algoritmo de ordenação, entretanto, não depende exclusivamente de sua complexidade assintótica, sendo também influenciada pelas propriedades da sequência de entrada [Estivill-Castro and Wood 1992]. Nesse contexto, a inversão, definida como um par de elementos que satisfaz $A[i] > A[j]$ para $i < j$, constitui uma medida quantitativa do grau de desordem de um arranjo [Knuth 1973, Mannila 1984]. O número de inversões permite caracterizar diferentes níveis de ordenação prévia (*presortedness*), uma vez que sequências com maior quantidade de inversões apresentam maior desordem em relação à ordem final [Mannila 1984]. Essa característica pode influenciar significativamente o custo de execução de determinados algoritmos, especialmente daqueles cuja quantidade de operações depende do grau de desordem da entrada [Estivill-Castro and Wood 1992].

Apesar dessa relação entre desordem da entrada e custo de execução, algoritmos quadráticos podem apresentar baixa capacidade de adaptação às condições iniciais dos dados [Mannila 1984, Estivill-Castro and Wood 1992]. Em particular, entradas com elevada quantidade de inversões podem aumentar o número de movimentações necessárias durante a ordenação, contribuindo para a degradação do desempenho de métodos como o InsertionSort [Cormen et al. 2022]. Nesse cenário, uma etapa preliminar capaz de reduzir a desordem da entrada pode diminuir o trabalho realizado posteriormente pelo algoritmo de ordenação. Surge, assim, a seguinte questão de pesquisa: em que medida a implementação de uma etapa de pré-processamento voltada à redução de inversões impacta o desempenho prático de algoritmos de ordenação quadráticos e quasilineares?

Formalmente, denota-se por C_{original} o custo computacional temporal da ordenação direta da entrada. Ao introduzir o pré-processamento simétrico com custo C_{pre} , o algoritmo de ordenação subsequente passa a atuar sobre um vetor modificado, despendendo um tempo reduzido C_{sort} . O objetivo deste trabalho é propor, implementar e avaliar um algoritmo simétrico de pré-processamento de baixo custo $O(n)$, projetado para reduzir o número de inversões de um vetor de entrada. Busca-se verificar experimentalmente se a abordagem híbrida apresenta maior eficiência temporal que a aplicação direta dos algoritmos convencionais, identificando rigorosamente as condições teóricas e práticas nas quais a condição $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$ é satisfeita.

A relevância deste estudo está na investigação de uma estratégia capaz de melhorar o comportamento prático de algoritmos de ordenação sem modificar sua estrutura fundamental ou introduzir elevado custo computacional adicional. A redução do grau de desordem antes da ordenação principal pode diminuir o número de operações realizadas em entradas desfavoráveis, especialmente em métodos sensíveis à quantidade de inversões [Estivill-Castro and Wood 1992, Hwang et al. 2000]. Além disso, o estudo contribui para a análise de algoritmos adaptativos ao investigar as condições nas quais o custo do pré-processamento é compensado pela redução do trabalho realizado durante a ordenação [Mannila 1984, Hwang et al. 2000].

A pesquisa adota uma abordagem quantitativa, aplicada e experimental [Gil 2019, Sampieri et al. 2021]. O estudo compreende a modelagem formal do algoritmo de pré-processamento e sua implementação em Rust. A avaliação experimental foi realizada por meio de *benchmarks* automatizados com a biblioteca Criterion, utilizando vetores sintéticos com tamanhos variando de 1.000 a 20.000 elementos e níveis

controlados de inversão, abrangendo situações de melhor caso, pior caso e ordenação parcial. Os experimentos compararam o tempo de execução da abordagem híbrida com o dos algoritmos de ordenação aplicados diretamente às mesmas entradas.

Este artigo está estruturado em cinco seções, além desta introdução. A Seção 2 apresenta o Referencial Teórico e a formalização do algoritmo de pré-processamento proposto. A Seção 3 descreve a Metodologia e os procedimentos experimentais. A Seção 4 apresenta e discute os resultados obtidos. Por fim, a Seção 5 apresenta as Considerações Finais.

2. Fundamentação Teórica

Nesta seção, serão abordados os conceitos fundamentais que sustentam a presente pesquisa, proporcionando uma base teórica sólida para a compreensão do impacto do pré-processamento simétrico sobre o desempenho de algoritmos de ordenação. Serão explorados tópicos como a medição da desordem por meio de inversões, os fundamentos da ordenação adaptativa, as características dos algoritmos avaliados (tanto quadráticos quanto quasilineares) e a formalização do algoritmo de pré-processamento simétrico proposto. O objetivo é contextualizar o leitor com os pilares teóricos necessários para analisar como a reestruturação prévia dos dados afeta o custo computacional das etapas subsequentes de ordenação.

2.1. Inversões e Ordenação Adaptativa

A eficiência prática dos algoritmos de ordenação tradicionais frequentemente depende da disposição inicial dos elementos no conjunto de dados de entrada [Cormen et al. 2022, Estivill-Castro and Wood 1992]. Para formalizar e quantificar o grau de desordem de uma sequência, utiliza-se classicamente o conceito de *inversão* [Knuth 1973]. Dado um vetor a composto por n elementos, uma inversão é definida formalmente como um par de índices (i, j) tal que $i < j$ e $a[i] > a[j]$ [Knuth 1973, Sedgewick and Wayne 2011]. O número total de inversões I constitui uma medida do grau de desordem da entrada e está diretamente relacionado ao esforço necessário para ordená-la por meio de operações locais [Mannila 1984]. A Figura 1 ilustra visualmente a identificação de inversões em um vetor arbitrário.

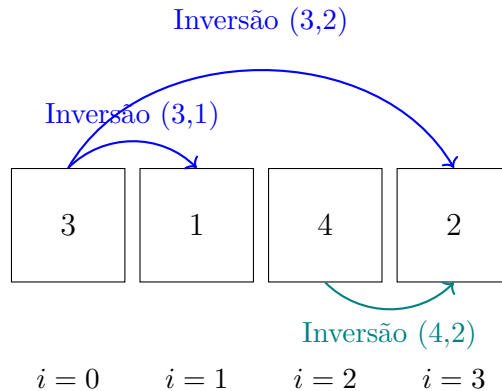


Figura 1. Representação visual do conceito de inversão em um vetor $a = [3, 1, 4, 2]$. O número total de inversões é $I = 3$, correspondente aos pares ordenados onde elementos maiores antecedem elementos menores em posições anteriores.

A relação entre o volume de inversões I e a complexidade de execução dá origem à propriedade de *adaptabilidade* em algoritmos de ordenação [Estivill-Castro and Wood 1992]. Um algoritmo é considerado adaptativo quando sua complexidade temporal melhora sensivelmente na presença de um ordenamento prévio parcial na entrada [Mannila 1984]. Um exemplo clássico é a ordenação por inserção (*insertion sort*), cuja execução consome $O(n + I)$ operações. Em um vetor estritamente ordenado ($I = 0$), o algoritmo atinge tempo linear $O(n)$, enquanto, no pior caso ($I = O(n^2)$), degrada para tempo quadrático [Cormen et al. 2022, Knuth 1973].

2.2. Algoritmos Avaliados

Para avaliar o impacto do pré-processamento simétrico sobre diferentes paradigmas de ordenação, selecionou-se um conjunto representativo de algoritmos com características distintas quanto à complexidade temporal, adaptabilidade e sensibilidade à ordem inicial dos elementos. A seleção foi realizada de modo a contemplar tanto algoritmos cuja eficiência pode ser diretamente influenciada pela quantidade de desordem presente na entrada quanto métodos cujo comportamento é menos dependente dessa propriedade.

No grupo de algoritmos com complexidade temporal $O(n^2)$ no pior caso, foram avaliados o *bubble sort*, o *insertion sort* e o *selection sort* [Cormen et al. 2022]. Os dois primeiros apresentam comportamento adaptativo, reduzindo o número de operações conforme o grau de ordenamento prévio da entrada [Estivill-Castro and Wood 1992], o que os torna relevantes para analisar os impactos da redução de inversões. Em contrapartida, o *selection sort* é não adaptativo e mantém um número fixo de comparações independentemente da configuração inicial [Estivill-Castro and Wood 1992], servindo como contraponto aos algoritmos sensíveis ao pré-processamento.

No grupo dos algoritmos de complexidade quasilinear, foram avaliados o *merge sort*, cuja complexidade temporal é $O(n \log n)$, e o *quicksort*, cujo desempenho médio é $O(n \log n)$, embora seu pior caso seja $O(n^2)$ [Cormen et al. 2022]. A inclusão desses métodos permite verificar se os efeitos observados nos algoritmos quadráticos permanecem relevantes quando o custo assintótico da ordenação é inferior. Dessa forma, o experimento não se restringe a algoritmos diretamente relacionados à contagem de inversões, permitindo analisar também a influência do pré-processamento em métodos cuja estratégia de ordenação apresenta características distintas.

Assim, o conjunto avaliado contempla três dimensões relevantes para a análise experimental: algoritmos quadráticos adaptativos (*bubble sort* e *insertion sort*), um algoritmo quadrático não adaptativo (*selection sort*) e algoritmos quasilineares baseados em estratégias distintas de ordenação (*merge sort* e *quicksort*). Essa composição permite verificar não apenas se o pré-processamento simétrico reduz a quantidade de inversões da entrada, mas também em que medida essa redução se converte em ganhos de desempenho para algoritmos com diferentes graus de dependência em relação à ordem inicial dos dados.

2.3. Pré-Processamento Simétrico

O conceito de pré-processamento, no contexto do projeto e análise de algoritmos, refere-se a uma etapa preliminar de transformação ou reestruturação dos dados de

entrada realizada antes da execução do algoritmo principal [Cormen et al. 2022]. O propósito dessa fase preparatória é alterar a disposição ou as propriedades da instância do problema, tornando a resolução subsequente mais simples ou computacionalmente mais eficiente. De forma conceitual e intuitiva, o pré-processamento atua de maneira análoga à organização prévia de um ambiente ou material de trabalho antes da execução de uma tarefa complexa: embora a etapa de preparação exija um custo computacional inicial, a redução no esforço exigido no processamento principal compensa esse investimento prévio.

Para que uma etapa de pré-processamento seja viável em problemas de ordenação, seu custo temporal deve ser estritamente dominado pela complexidade do algoritmo de ordenação que o sucede [Cormen et al. 2022, Sedgewick and Wayne 2011]. Dessa forma, um pré-processamento ideal deve apresentar complexidade temporal linear $O(n)$, garantindo que o *overhead* introduzido seja assintoticamente desprezível em relação ao custo total da ordenação — seja em algoritmos quasilineares $O(n \log n)$ no caso médio ou quadráticos $O(n^2)$ no pior caso [Knuth 1973].

Especificamente nesta pesquisa, a etapa de pré-processamento é denominada *simétrica* por aplicar uma varredura bidirecional e espelhada a partir das extremidades do vetor. Essa reestruturação visa reduzir o número total de inversões I da entrada ao mover elementos discrepantes para posições mais próximas de seus destinos finais em um único passe de complexidade $O(n)$. Como resultado, ao fornecer uma estrutura parcialmente reordenada para os algoritmos subsequentes, busca-se mitigar o número de comparações e trocas operadas tanto por métodos adaptativos quanto por métodos não adaptativos.

3. Metodologia

Esta seção detalha a abordagem metodológica adotada para a realização deste trabalho, descrevendo a caracterização da pesquisa, a configuração do ambiente experimental e os procedimentos empregados no benchmarking dos algoritmos. A investigação foi conduzida por meio de um protocolo experimental controlado, utilizando execuções compiladas em linguagem Rust e métricas estatísticas rigorosas fornecidas pela biblioteca *Criterion*. Para assegurar a abrangência dos testes e a replicabilidade dos resultados, foram projetadas seis topologias distintas de vetores sob controle de semente aleatória. A escolha desta metodologia visa garantir a precisão temporal e a validade interna da avaliação do pré-processamento simétrico, fornecendo um caminho metodológico transparente e totalmente reprodutível.

3.1. Tipo de Pesquisa e Configuração

O presente estudo caracteriza-se como uma pesquisa aplicada, quantitativa, explicativa e experimental [Gil 2019, Sampieri et al. 2021]. A caracterização apoia-se no isolamento e na manipulação controlada da topologia das entradas para medir a variação no tempo de execução e na contagem de inversões.

Os experimentos foram executados em ambiente controlado sob o sistema operacional Windows 11 Pro (64-bit, build 10.0.26200), em estação de trabalho equipada com processador AMD Ryzen 7 8700G (8 núcleos, 16 threads, *clock* base de 4,2 GHz) e 16 GB de memória RAM DDR5 a 5600MT/s. Durante os testes, o plano

de energia do sistema operacional permaneceu fixado em alto desempenho, com os processos de segundo plano minimizados para mitigar ruídos de escalonamento do sistema.

Todo o ecossistema foi desenvolvido utilizando a linguagem Rust 1.96.0 (*target x86_64-pc-windows-msvc*), compilado com a flag de otimização máxima (*release profile*, `opt-level = 3`). A tabela 1 detalha o ambiente e os parâmetros operacionais.

Tabela 1. Ambiente experimental e ferramentas utilizadas.

Componente	Configuração / Especificação
Sistema operacional	Windows 11 Pro (build 10.0.26200)
Processador	AMD Ryzen 7 8700G (8 núcleos / 16 threads)
Memória	16 GB DDR5 5600MT/s
Linguagem	Rust 1.96.0 (perfil <i>release</i>)
Ferramentas de Medição	Criterion 0.5 e Ferramenta CLI do Projeto
Geração de dados	Seed 42 (<i>ChaCha8Rng</i>), entradas pareadas

Para avaliar a resiliência e a adaptabilidade dos algoritmos sob diferentes graus de entropia, foram projetados seis padrões estruturais de vetores de entrada (Tabela 2), avaliados nas instâncias de tamanho $n \in \{1.000, 5.000, 10.000, 20.000\}$. A aleatoriedade determinística foi assegurada pelo pareamento rigoroso das sementes de geração entre as abordagens.

Tabela 2. Conjuntos de dados e caracterização das topologias de entrada.

Topologia	Descrição do Padrão Estrutural
Aleatório	Permutação aleatória uniforme de n valores distintos
Tartarugas	Primeira metade com valores elevados; segunda metade com valores baixos
Zigue-zague	Alternância sistemática de blocos crescentes e decrescentes
Quase ordenado	Vetor ordenado submetido a $n/100$ trocas aleatórias de pares
Duplicados	Vetor com baixa cardinalidade de chaves (valores inteiros no intervalo 0–2)
Invertido	Vetor arranjado em ordem estritamente decrescente (pior caso teórico)

3.2. Protocolo Experimental e Raciocínio de Cálculo de Inversões

O protocolo de medição empírica foi desenhado para isolar interferências do ambiente e garantir a reprodutibilidade estatística das métricas temporais. A quantificação do tempo de execução utilizou o arcabouço *Criterion.rs*, que executa ciclos automatizados de aquecimento de memória cache (*warm-up*) e realiza reamostragens estatísticas (*bootstrapping*) para mitigar o impacto de interrupções de hardware. Para cada combinação de algoritmo, topologia e tamanho de vetor n , foram calculados o tempo médio, a mediana e os intervalos de confiança de 95%. A medição temporal das abordagens submetidas ao pré-processamento contabilizou rigorosamente o custo composto $C_{\text{total}} = C_{\text{pre}} + C_{\text{sort}}$, garantindo a aferição transparente da condição de viabilidade prática $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$.

Adicionalmente, na quantificação experimental do número exato de inversões I , os testes preliminares adotaram uma rotina de checagem direta $O(n^2)$. Todavia, reconhece-se o gargalo metodológico associado a essa contagem no processo

experimental estendido. O raciocínio computacional para contagem de inversões pode ser otimizado para complexidade $O(n \log n)$ por meio do algoritmo MergeSort modificado [Cormen et al. 2022] ou da utilização de uma Árvore Fenwick (*Binary Indexed Tree* – BIT) [Fenwick 1994].

Na abordagem por MergeSort, durante a etapa de intercalação de dois subvetores ordenados L e R , quando um elemento $R[j]$ é menor que $L[i]$, adiciona-se exatamente $|L| - i$ ao contador de inversões [Cormen et al. 2022]. Essa substituição metodológica remove o gargalo quadrático associado à contagem de inversões, ampliando significativamente a capacidade do benchmark de validação e possibilitando, em trabalhos futuros, a avaliação de vetores com ordens de grandeza de 10^5 a 10^6 elementos.

3.3. Formalização do Pré-Processamento Simétrico

O procedimento de pré-processamento simétrico proposto opera como uma etapa determinística de varredura única sobre o vetor de entrada A de tamanho n . O objetivo desta etapa é reestruturar a disposição dos elementos através de trocas espelhadas e ajustes locais, antecedendo a execução dos algoritmos de ordenação avaliados. A estrutura procedural e as regras de controle do algoritmo estão formalizadas no Algoritmo 1.

Algoritmo 1: Algoritmo de Pré-Processamento Simétrico

Input: Vetor A de tamanho n
Output: Vetor A parcialmente reorganizado

$n \leftarrow$ tamanho de A ;
if $n < 2$ **then**
 | **return**;
end
 $ultimo \leftarrow n - 1$;
 $meio \leftarrow \lfloor n/2 \rfloor$;
for $i \leftarrow 0$ **to** $meio - 1$ **do**
 | $j \leftarrow ultimo - i$;
 | **if** $A[i] > A[j]$ **then**
 | | trocar($A[i]$, $A[j]$);
 | **end**
 | **if** $i + 1 < meio$ **then**
 | | **if** $A[i] > A[i + 1]$ **then**
 | | | trocar($A[i]$, $A[i + 1]$);
 | | **end**
 | **end**
 | **if** $j > 0$ e $j - 1 > meio$ **then**
 | | **if** $A[j] < A[j - 1]$ **then**
 | | | trocar($A[j]$, $A[j - 1]$);
 | | **end**
 | **end**
end

O algoritmo itera sobre a primeira metade do vetor, varrendo os índices

$i \in [0, \lfloor n/2 \rfloor - 1]$. A cada iteração, calcula-se o índice simétrico correspondente no extremo oposto do vetor, dado por $j = (n - 1) - i$. A execução do laço é composta por três verificações condicionais estruturadas:

1. **Troca Simétrica Extrema:** Avalia-se a relação de ordem entre as extremidades opostas. Se $A[i] > A[j]$, efetua-se a troca direta dos elementos, posicionando o menor elemento no segmento esquerdo e o maior no segmento direito.
2. **Ajuste Adjacente Esquerdo:** Para os índices em que $(i + 1) < \lfloor n/2 \rfloor$, verifica-se a ordenação local na metade esquerda. Se $A[i] > A[i + 1]$, realiza-se a troca entre os elementos adjacentes.
3. **Ajuste Adjacente Direito:** Para os índices em que $(j - 1) > \lfloor n/2 \rfloor$ e $j > 0$, verifica-se a ordenação local na metade direita. Se $A[j] < A[j - 1]$, executa-se a troca entre os elementos adjacentes.

As condições de contorno garantem que as trocas adjacentes operem estritamente dentro de seus respectivos subdomínios (esquerdo e direito), sem ultrapassar o ponto médio do vetor. Em termos de complexidade computacional, o algoritmo executa exatamente $\lfloor n/2 \rfloor$ iterações. Em cada passo do laço, realiza-se um número constante de comparações (no máximo 3) e trocas (no máximo 3), resultando em uma complexidade temporal de pior caso $T(n) = O(n)$. Como a reestruturação dos dados é realizada *in-place* no próprio vetor A , a complexidade espacial auxiliar é estritamente $O(1)$.

3.4. Ameaças à Validade

As medições de tempo físico foram protegidas contra ruídos por meio do aquecimento prévio da CPU e da alternância da ordem de execução. A ausência de estatísticas de baixo nível de hardware, como *cache misses*, decorre da indisponibilidade da interface `perf_event_open`, específica do Linux, no ambiente Windows utilizado. Todos os experimentos podem ser reproduzidos por meio do comando `cargo bench` e das rotinas disponibilizadas no repositório do projeto [Nascimento 2026].

4. Resultados e Discussão

Esta seção apresenta a análise e discussão dos resultados experimentais obtidos, avaliando o impacto do pré-processamento simétrico sobre a estrutura dos dados e o tempo de execução dos algoritmos. A discussão segue uma estrutura analítica que parte do impacto direto na redução de inversões e da interpretação do viés geométrico observado no cenário invertido, avança para a mensuração do tempo total de ordenação em algoritmos quadráticos e quasilineares, e culmina na interpretação dos efeitos microarquiteturais, como a localidade de referência e o comportamento das caches de CPU. O objetivo é demonstrar de forma consolidada os cenários em que a técnica oferece ganhos reais de desempenho e entender as limitações físicas e algorítmicas envolvidas.

4.1. Redução de Inversões e Viés Estrutural do Caso Invertido

A Figura 2 ilustra a variação das inversões observadas nos vetores de 10.000 elementos, cujos valores quantitativos exatos e porcentagens de redução estão detalhados na

Tabela 3. Observa-se que o pré-processamento é capaz de mitigar expressivamente a desordem em múltiplos cenários. Contudo, a interpretação da eliminação total das inversões (100%) no cenário *Invertido* exige sobriedade teórica e analítica.

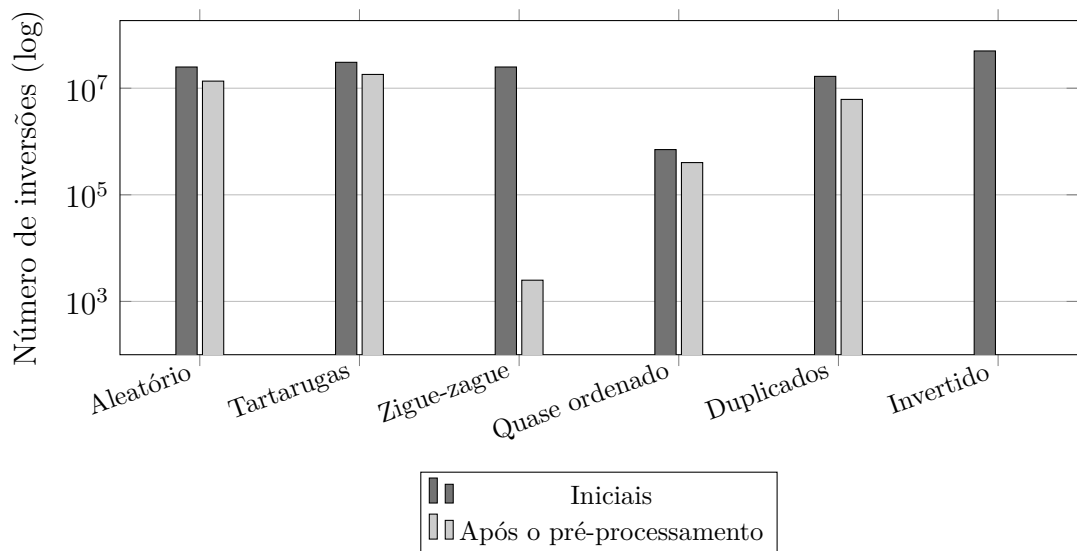


Figura 2. Número de inversões antes e após o pré-processamento simétrico para vetores de 10.000 elementos (escala logarítmica).

Tabela 3. Redução de inversões proporcionada pelo pré-processamento simétrico em vetores de 10.000 elementos.

Tipo	Inversões	Pós-pré	Redução
Aleatório	24.993.860	13.586.692	45,6%
Tartarugas	30.613.750	18.133.750	40,8%
Zigue-zague	24.997.500	2.501	99,9%
Quase ordenado	705.478	402.318	43,0%
Duplicados	16.678.243	6.166.988	63,0%
Invertido	49.995.000	0	100,0%

Como o Algoritmo 1 executa a troca direta entre $a[i]$ e $a[n - 1 - i]$, um vetor estritamente invertido (onde $a[i] > a[n - 1 - i]$ para todo $i < n/2$) sofre uma reversão geométrica completa da sua sequência em $O(n)$ operações. A resolução do caso Invertido decorre da geometria da transformação do pré-processamento, configurando um caso ideal onde a topologia da desordem coincide perfeitamente com a simetria aplicada.

4.2. Impacto no Tempo de Execução e Custo do Pré-processamento

A Tabela 4 apresenta o tempo médio de execução e o ganho relativo obtido por cada algoritmo sob vetores de 10.000 elementos, evidenciando como a eficiência do pré-processamento é condicionada pela complexidade e adaptabilidade nativas de cada método. Os algoritmos quadráticos sensíveis à ordenação (como *Inserção* e *Bubblesort*) obtiveram os maiores benefícios práticos: no cenário *Zigue-zague*, o pré-processamento reduziu o tempo do *Inserção* de 5.062,86 μ s para 11,98 μ s

(+99,8%) e do *Bubblesort* de 19.829,76 μs para 17,48 μs (+99,9%), refletindo a eliminação drástica das inversões. Em contrapartida, métodos de complexidade assintótica $O(n \log n)$, como o *Merge*, apresentaram variações marginais (entre $-14,0\%$ e $+14,5\%$). Como a estrutura de divisão e conquista já garante alta eficiência nativa, o custo computacional da etapa prévia tende a suplantiar eventuais ganhos em distribuições de baixa estrutura.

Tabela 4. Tempo médio por execução (em μs , média $\pm$ meia-largura do IC 95%) e ganho do pré-processamento para vetores de 10.000 elementos. Fonte: Criterion, seed 42.

Algoritmo	Tipo	Puro	Com pré	Ganho
Merge	Aleatório	416,90 $\pm$ 7,82	429,00 $\pm$ 5,30	-2,9%
Merge	Tartarugas	116,11 $\pm$ 1,92	132,34 $\pm$ 1,79	-14,0%
Merge	Zigue-zague	128,08 $\pm$ 1,88	109,53 $\pm$ 2,08	+14,5%
Merge	Quase ordenado	128,72 $\pm$ 2,66	134,03 $\pm$ 1,86	-4,1%
Merge	Duplicados	191,41 $\pm$ 2,89	193,68 $\pm$ 3,84	-1,2%
Merge	Invertido	121,78 $\pm$ 2,87	108,17 $\pm$ 2,56	+11,2%
Quicksort	Aleatório	334,51 $\pm$ 6,68	351,30 $\pm$ 5,57	-5,0%
Quicksort	Tartarugas	3240,29 $\pm$ 19,06	801,36 $\pm$ 16,53	+75,3%
Quicksort	Zigue-zague	683,60 $\pm$ 16,17	316,25 $\pm$ 5,13	+53,7%
Quicksort	Quase ordenado	242,49 $\pm$ 4,45	265,87 $\pm$ 5,23	-9,6%
Quicksort	Duplicados	9815,64 $\pm$ 76,56	9929,86 $\pm$ 43,56	-1,2%
Quicksort	Invertido	144,62 $\pm$ 3,38	73,87 $\pm$ 1,51	+48,9%
Inserção	Aleatório	5065,81 $\pm$ 37,71	2881,55 $\pm$ 26,64	+43,1%
Inserção	Tartarugas	6131,06 $\pm$ 37,49	3643,55 $\pm$ 16,46	+40,6%
Inserção	Zigue-zague	5062,86 $\pm$ 17,57	11,98 $\pm$ 0,29	+99,8%
Inserção	Quase ordenado	219,59 $\pm$ 5,18	152,59 $\pm$ 2,39	+30,5%
Inserção	Duplicados	3354,86 $\pm$ 20,24	1408,90 $\pm$ 48,91	+58,0%
Inserção	Invertido	9943,20 $\pm$ 39,46	11,15 $\pm$ 0,20	+99,9%
Bubblesort	Aleatório	29011,57 $\pm$ 96,34	24958,26 $\pm$ 55,31	+14,0%
Bubblesort	Tartarugas	19917,67 $\pm$ 88,28	15009,86 $\pm$ 57,43	+24,6%
Bubblesort	Zigue-zague	19829,76 $\pm$ 57,13	17,48 $\pm$ 0,46	+99,9%
Bubblesort	Quase ordenado	20518,06 $\pm$ 54,61	17571,95 $\pm$ 32,00	+14,4%
Bubblesort	Duplicados	25626,62 $\pm$ 79,13	21119,02 $\pm$ 40,00	+17,6%
Bubblesort	Invertido	19816,65 $\pm$ 84,20	9,17 $\pm$ 0,22	+100,0%
Seleção	Aleatório	24484,27 $\pm$ 851,48	23492,52 $\pm$ 772,67	+4,1%
Seleção	Tartarugas	24705,38 $\pm$ 895,32	24664,67 $\pm$ 739,03	+0,2%
Seleção	Zigue-zague	25396,38 $\pm$ 827,27	19792,89 $\pm$ 57,95	+22,1%
Seleção	Quase ordenado	25505,03 $\pm$ 847,40	24613,47 $\pm$ 813,60	+3,5%
Seleção	Duplicados	23435,17 $\pm$ 852,87	25383,85 $\pm$ 833,81	-8,3%
Seleção	Invertido	17686,34 $\pm$ 120,06	19780,12 $\pm$ 54,58	-11,8%

A viabilidade prática dessa abordagem fundamenta-se no baixo e altamente previsível custo computacional da etapa simétrica. Conforme demonstrado na Tabela 5, o tempo de execução do pré-processamento apresenta um comportamento estritamente linear $O(n)$ em relação ao tamanho do vetor, evoluindo de 1,42 μs para $n = 1.000$ até 24,00 μs para $n = 20.000$. Notadamente, a métrica de tempo unitário reduz-se de 1,42 ns para estabilizar-se na faixa de 1,20 a 1,24 ns por

elemento a partir de $n \geq 5.000$. Esse padrão indica a ausência de gargalos de localidade de referência na memória e confirma que a varredura linear impõe um *overhead* irrisório — da ordem de apenas $12,17 \mu s$ em vetores de 10.000 elementos —, perfeitamente compensado pela economia de milissegundos obtida na fase de ordenação de algoritmos adaptativos.

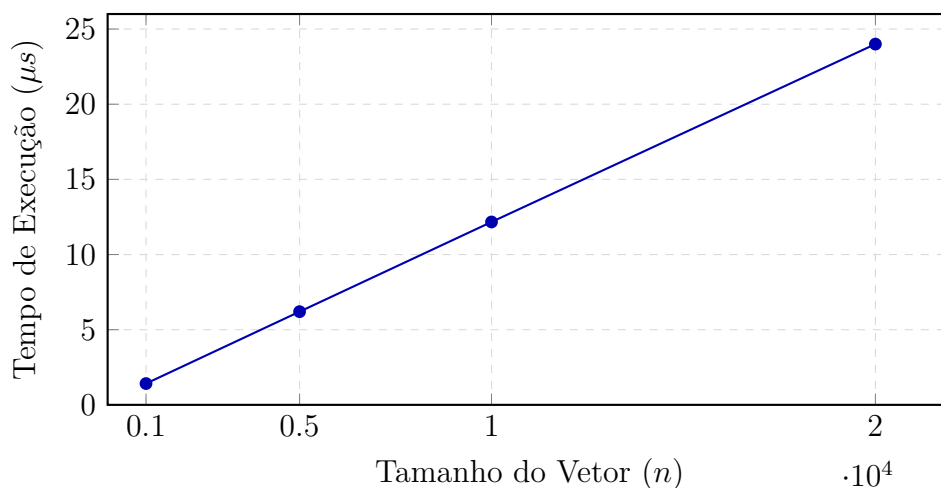


Figura 3. Tempo médio de execução do pré-processamento em função da dimensão do vetor (n), evidenciando comportamento linear $O(n)$.

Tabela 5. Custo médio do pré-processamento simétrico em função do tamanho do vetor (média sobre os seis tipos).

Tamanho	Pré (μs)	ns/elemento
1.000	1,42	1,42
5.000	6,20	1,24
10.000	12,17	1,22
20.000	24,00	1,20

4.3. Tempo Total de Ordenação e Análise de Localidade de Caches

A avaliação do tempo total de execução revela uma acentuada dicotomia no impacto do pré-processamento, cujo êxito está diretamente correlacionado à complexidade nativa e à adaptabilidade estrutural dos métodos avaliados. Nos algoritmos adaptativos de classe quadrática, notadamente o *InsertionSort* e o *BubbleSort*, a drástica eliminação prévia das inversões resulta em ganhos substanciais de desempenho (variando entre 30% e 99,9%). Esse comportamento é justificado pela matemática do algoritmo: o modesto custo linear da varredura simétrica ($\approx 1,2$ ns/elemento) é vastamente amortizado pela supressão das operações de trocas locais, altamente custosas na ordem $O(n^2)$.

Por outro lado, métodos com complexidade estrutural inerente de $O(n \log n)$, a exemplo do *Quicksort* operando sobre vetores puramente aleatórios, exibem uma leve degradação de desempenho na abordagem híbrida. Embora a etapa simétrica efetivamente reduza o número absoluto de inversões, esse ganho teórico não se converte em aceleração prática, uma vez que a divisão e conquista já lida de forma

otimizada com a desordem inicial. A principal justificativa arquitetural para esse *overhead* reside na localidade espacial de caches.

Ao acessar e manipular simultaneamente posições nas duas extremidades do vetor, o pré-processamento simétrico impõe um padrão de acesso bifronte à memória principal. Esse comportamento diverge da varredura sequencial tradicional, o que reduz a eficácia dos mecanismos de pré-busca (*prefetching*) de hardware e introduz latências adicionais decorrentes de possíveis *cache misses* [Patterson and Hennessy 2012]. Consequentemente, em cenários onde o ganho proporcionado pela redução de inversões é marginal, os custos arquiteturais de acesso à memória superam os benefícios lógicos da etapa.

Toda essa dinâmica — o contraste entre o expressivo aproveitamento lógicos dos métodos quadráticos e a limitação de *hardware* nos algoritmos $O(n \log n)$ — encontra-se visualmente consolidada na Figura 4, que detalha o tempo médio de execução puro contraposto ao cenário com pré-processamento para todos os perfis de desordem sob $n = 10.000$.

5. Considerações Finais

Este trabalho apresentou e avaliou um pré-processamento simétrico de complexidade $O(n)$ destinado à redução do número de inversões em sequências de entrada para algoritmos de ordenação. Os resultados demonstraram que a abordagem pode proporcionar ganhos relevantes em algoritmos adaptativos de complexidade quadrática quando aplicada a entradas com padrões estruturados de desordem ou em ordem inversa. Esse ganho é expressivo especialmente quando a redução do custo de ordenação supera o custo adicional introduzido pela etapa preliminar, satisfazendo a condição $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$.

Por outro lado, nos algoritmos de complexidade $O(n \log n)$ avaliados, a aplicação do método revelou limitações importantes. Particularmente em entradas aleatórias, o custo adicional do pré-processamento e seus possíveis efeitos adversos sobre a localidade de cache não compensaram os benefícios obtidos com a redução de inversões. Tal constatação indica que a viabilidade prática da técnica deve ser estritamente condicionada às características da distribuição da entrada e à classe do algoritmo de ordenação empregado.

Como trabalhos futuros, propõe-se substituir a estratégia experimental utilizada para a contagem de inversões por uma estrutura baseada em árvore Fenwick, de complexidade $O(n \log n)$, o que permitirá ampliar a escala dos experimentos para entradas superiores a 1.000.000 de elementos. Adicionalmente, recomenda-se a realização de testes em ambiente Linux com a infraestrutura `perf_event`, viabilizando a análise direta de eventos de hardware, como *cache misses* e *branch mispredictions*. Essas extensões possibilitarão investigar detalhadamente os impactos da técnica sobre a hierarquia de memória e a execução especulativa.

Referências

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, 4th edition.

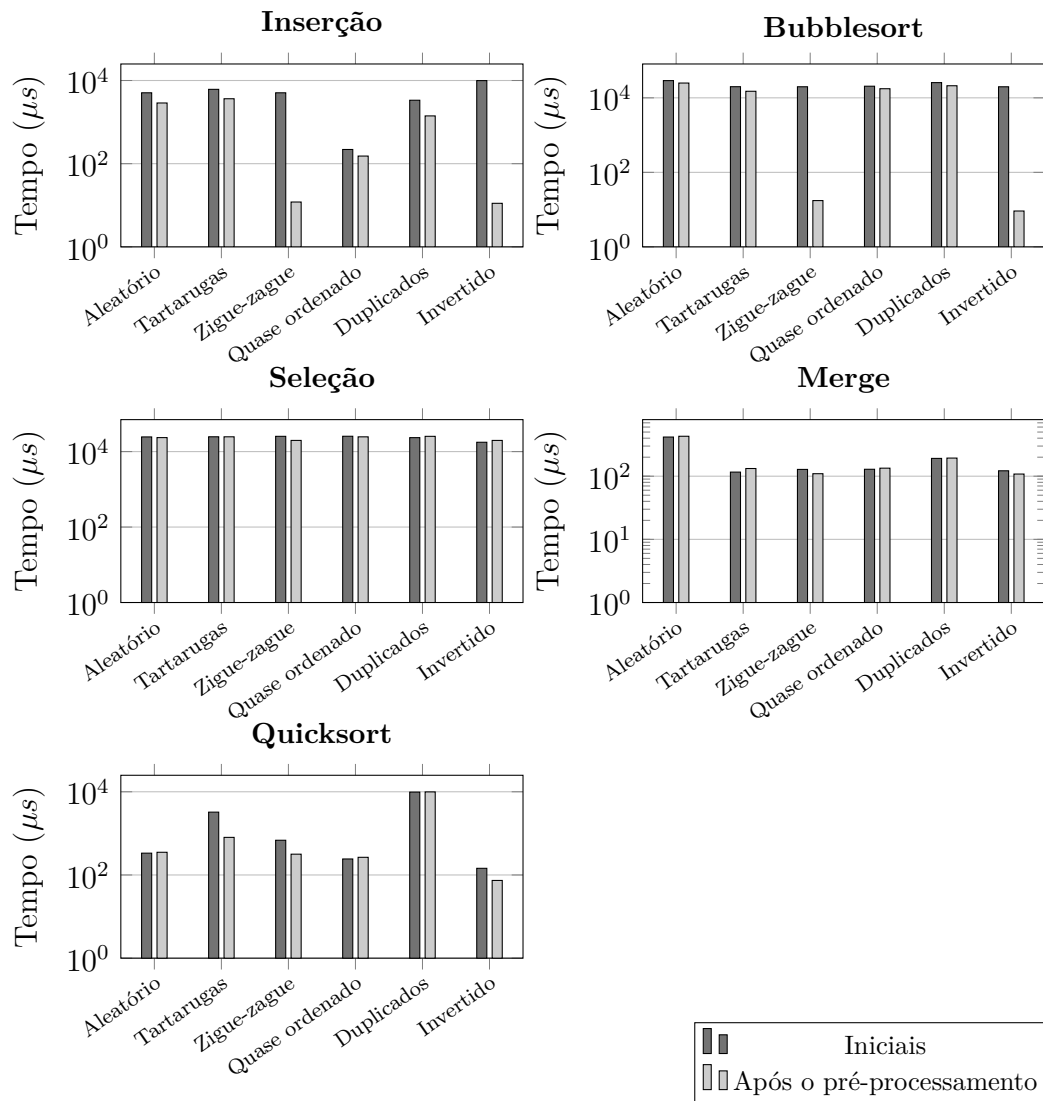


Figura 4. Comparação temporal entre a ordenação pura e com pré-processamento simétrico ($n = 10.000$).

- Estivill-Castro, V. and Wood, D. (1992). A survey of adaptive sorting algorithms. *ACM Computing Surveys*, 24(4):441–476.
- Fenwick, P. M. (1994). A new data structure for cumulative frequency tables. *Software: Practice and Experience*, 24.
- Gil, A. C. (2019). *Métodos e técnicas de pesquisa social*. Editora Atlas Ltda.
- Hwang, H.-K., Yang, B.-Y., and Yeh, Y.-N. (2000). Presorting algorithms: An average-case point of view. *Theoretical Computer Science*, 242(1-2):29–40.
- Knuth, D. E. (1973). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA.
- Mannila, H. (1984). Measures of presortedness and optimal sorting algorithms: Extended abstract. In Goos, G., Hartmanis, J., Barstow, D., Brauer, W., Brinch Hansen, P., Gries, D., Luckham, D., Moler, C., Pnueli, A., Seegmüller, G., Stoer, J.,

- Wirth, N., and Paredaens, J., editors, *Automata, Languages and Programming*, volume 172 of *Lecture Notes in Computer Science*, pages 324–336. Springer Berlin Heidelberg, Berlin, Heidelberg.
- Nascimento, S. d. P. (2026). Presorting algorithm. Disponível em: <https://github.com/Zidan-09/presorting-algorithm>. Repositório de código-fonte e dados experimentais.
- Patterson, D. A. and Hennessy, J. L. (2012). *Computer organization and design: the hardware/software interface*. The Morgan Kaufmann series in computer architecture and design. Elsevier/Morgan Kaufmann, Amsterdam Boston Heidelberg, revised fourth edition edition.
- Sampieri, R. H., Collado, C. F., and Lucio, M. d. P. B. (2021). *Metodologia de pesquisa*. Penso.
- Sedgewick, R. and Wayne, K. D. (2011). *Algorithms*. Addison-Wesley, Upper Saddle River, 4th edition.
- Ziviani, N. (2021). *Projeto De Algoritmos Com Implementações Em Java E C++*. Cengage Learning.